

Project Documentation: ServiceNow User Deletion Prevention

1. Project Overview

The objective of this project is to maintain data integrity and audit trails within ServiceNow by preventing the deletion of user records that are actively associated with incident tickets. Deleting a user who is currently "Assigned To" an incident can lead to "Empty" reference fields, breaking reporting metrics and losing historical accountability. This solution implements a **Business Rule** to intercept deletion attempts and enforce a validation check.

2. Functional Requirements

- **Trigger:** The system must check for active associations when a "Delete" action is initiated on a sys_user record.
 - **Validation:** If the user is found in the assigned_to field of any record in the incident table, the deletion must be blocked.
 - **User Feedback:** A clear error message must be displayed to the administrator explaining why the action was aborted.
 - **Execution:** The check must happen server-side to ensure it cannot be bypassed by UI-level actions.
-

3. Technical Implementation

3.1 Business Rule Configuration

To implement this logic, a Business Rule was created with the following configurations:

- **Table:** User [sys_user]
- **Advanced:** Checked
- **When:** Before
- **Order:** 100
- **Action:** Delete (Checked)

3.2 Logic Script

The following Server-Side Script executes whenever a deletion is attempted. It utilizes the GlideRecord API to query the Incident table.

JavaScript

```
(function executeRule(current, previous /*null when async*/) {  
  // Initialize a query on the Incident table  
  var incGr = new GlideRecord('incident');  
  
  // Check if the 'assigned_to' field matches the sys_id of the user being deleted  
  incGr.addQuery('assigned_to', current.sys_id);  
  
  // Efficiency Optimization: We only need to find one instance to block deletion  
  incGr.setLimit(1);  
  
  incGr.query();  
  
  // If a record is found, abort the process  
  if (incGr.next()) {  
    gs.addErrorMessage('This user cannot be deleted because they are assigned to one or  
more incidents.');
```

current.setAbortAction(true); // This stops the database deletion
 }
})(current, previous);

3.3 Workflow Logic

1. **Delete Trigger:** Admin clicks "Delete" on a User Profile.
2. **Query:** The script searches the incident table for assigned_to = [User's Sys_ID].
3. **Condition Met:** If a match is found, setAbortAction(true) is triggered.
4. **Result:** The user remains in the database, and a red error banner appears at the top of the screen.

4. Test Plan & Execution

Test Case 01: Assigned User Deletion (Negative Test)

- **User:** Andrew Och
- **Setup:** Ensure "Andrew Och" is assigned to at least one active Incident (e.g., INC0010001).
- **Action:** Navigate to the User record for Andrew Och and click **Delete**.
- **Expected Result:** System displays error: *"This user cannot be deleted because they are assigned to one or more incidents."*

- **Actual Result:** Deletion Aborted. User record remains.

Test Case 02: Unassigned User Deletion (Positive Test)

- **User:** [Test User Name]
 - **Setup:** Create a new user with no assigned incidents.
 - **Action:** Click **Delete**.
 - **Expected Result:** System successfully deletes the record.
 - **Actual Result:** Record deleted successfully.
-

5. Conclusion

This Business Rule effectively safeguards the ServiceNow environment against accidental data loss. By enforcing this referential integrity at the server level, we ensure that the Incident Management process remains traceable.

Future Recommendations: * Expand this rule to check other task tables (Change Requests, Problems, etc.).

- Consider implementing a "Soft Delete" (setting the user to active=false) instead of a hard deletion to preserve historical data.
-